[CSED233-01] Data Structure

Algorithm Analysis

Seung-Hwan Baek

POSTECH

Review: C++

- Compiler, linker
- Variable
- Array, pointer, dynamic memory
- Loop, break, continue

```
int a[10] = {1,2,-3,-4,-5,5,4,3,-2,-1};
int sum = 0;

for (int i = 0; i < 10; i++) {
    if (a[i] < 0) break;
    sum += a[i];
}

 cout << "sum : " << sum << endl;
```

Algorithm

- Algorithm is a step-by-step procedure for solving a problem in a finite amount of time
- This algorithm below sums the elements of an array until finding the first negative value
- Can you think about a better algorithm?

```
int a[10] = {1,2,-3,-4,-5,5,4,3,-2,-1};
int sum = 0;

for (int i = 0; i < 10; i++) {
    if (a[i] < 0) break;
    sum += a[i];
}

 cout << "sum : " << sum << endl;
```

Can we determine which algorithm is better?

Comparing Algorithms

- How to compare two algorithms?
- One measure is efficiency
- Efficiency in terms of running time
  - Time complexity
- Efficiency in terms of space requirements
  - Space complexity

Comparing Algorithms

- There are two ways of comparing two algorithms for a same problem

1. Empirical studies
2. Theoretical analysis

Empirical Studies

- Programming & testing
  - Write programs implementing the algorithms
  - Run the programs with varying inputs
  - Measure the runtimes and space requirements

- Limitations
  - Lots of effort to implement the algorithm
  - Results may not apply to other inputs which are not included in the experiment
  - For fair comparison, the same H/W and S/W environments must be used

Quiz

- What are the benefits of using empirical studies?

Theoretical Analysis

- Theoretical analysis focuses on the high-level description of the algorithm instead of an implementation
- Here, we consider the running time/space requirements as the functions of the input size n
- Usually, it considers all possible inputs

+ The theoretical analysis is independent of the hardware/software environments

Quiz

- What are the limitations of using theoretical analysis?

Asymptotic (Theoretical) Analysis

Asymptotic analysis is a method of describing limiting behavior

1. Lower-order terms are not important

- If $f(n) = n^2 + 3n$, then as n becomes very large, the term $3n$ becomes insignificant compared to n^2

The function $f(n)$ is said to be "asymptotically equivalent to n^2, as $n \to \infty$"

Asymptotic (Theoretical) Analysis

Asymptotic analysis is a method of describing limiting behavior

2. Constants do not matter

- Consider $6N^2$ and $20N^2$
- When $N >> 20$, N^2 is what is driving the function's increase
- The constants 6 and 20 are not that important

The function $6N^2$ is said to be "asymptotically equivalent to $20N^2$, as $n \rightarrow \infty$"

Best, Worst, and Average Cases

Different inputs of a given size can require different running times

1. Best case (input)
   - At least, takes this much
2. Average case (input)
   - Usually, it takes this much
   - Difficult to determine
3. Worst case (input)
   - It could take up to this
   - Easier to analyze
   - Crucial to interactive applications

Mostly, theoretical analysis focuses on the worst-case inputs

Asymptotic (Theoretical) Analysis

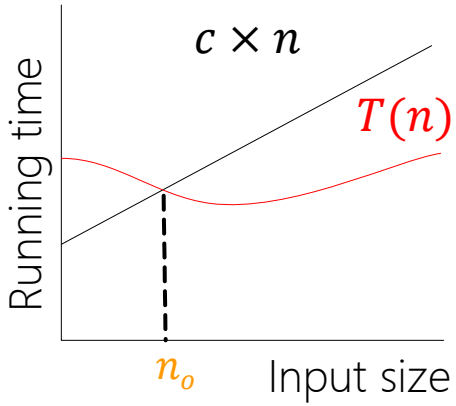
- Let $T(n)$ the running-time function of an algorithm that maps an input size n to the corresponding running time
- To capture the growth rate behavior of $T(n)$ in the long run, we use the following metric for each case

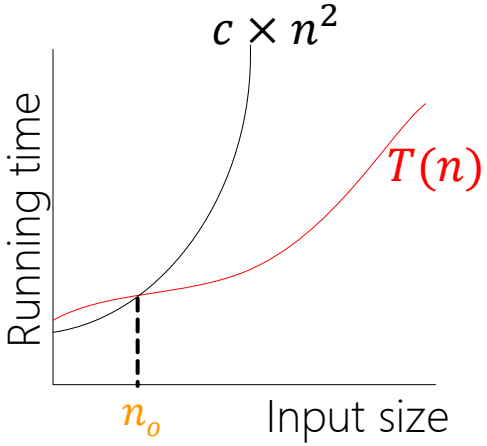
1. Worst case: Big-O
2. Average case: Big-Theta
3. Best : Big-Omega

Big-O Notation

An algorithm is $O(f(n))$
if there exist a constant $c > 0$ & an integer constant $n_0 \geq 1$
such that $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$

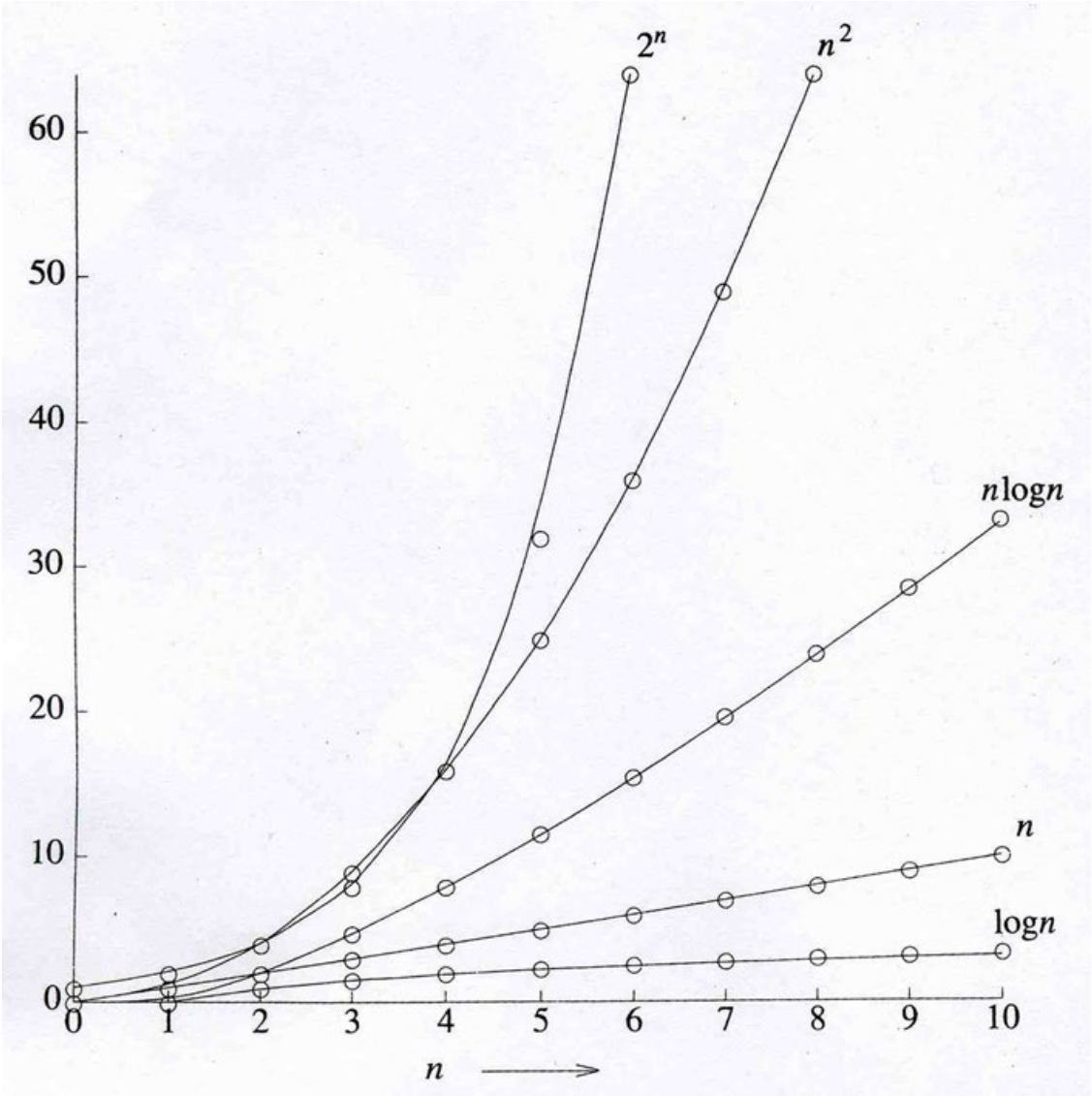
- Then, we write $T(n) \in O(f(n))$, or $T(n) = O(f(n))$
- Upper bound on the growth rate of $T(n)$





Typical Growth Rate

- $O(1)$ – constant
- $O(\log(n))$ – logarithmic
- $O(n)$ – linear
- $O(n\log(n))$ – log linear
- $O(n^2)$ – quadratic
- $O(n^3)$ – cubic
- $O(2^n)$ – exponential
- $O(n!)$ – factorial



Big-O Notation: Examples

- Example: $T(n) = (n+1)^2$ is $O(n^2)$

$$T(n) = (n+1)^2 = n^2 + 2n + 1$$
$$\leq n^2 + 2n^2 + n^2 = 4n^2 \text{ for all } n \geq 1$$
$$\text{Thus pick } c = 4 \text{ and } n_0 = 1$$

- More examples:

$$3n^3 \in O(n^3)\text{: tight bound} \Leftrightarrow 3n^3 \in O(n^4)\text{: loose bound}$$
$$3n^3 + 2n^2 + 8 \in O(n^3)$$
$$2^{100} \in O(1)$$
$$3\log(n) + 5 \in O(\log(n))$$

Properties of Big-O

- Addition rule:

$$T_1(n) \in O(f(n)) \text{ and } T_2(n) \in O(g(n)) \Rightarrow T_1(n) + T_2(n) \in O(\max\{f(n), g(n)\})$$

- Product rule:

$$T_1(n) \in O(f(n)) \text{ and } T_2(n) \in O(g(n)) \Rightarrow T_1(n) \cdot T_2(n) \in O(f(n) \cdot g(n))$$

- Others

$$\text{For any constant } a > 0,\ T(n) \in O(f(n)) \Rightarrow a \cdot T(n) \in O(f(n))$$

$$T(n) \in O(f(n)) \text{ and } f(n) \in O(g(n)) \Rightarrow T(n) \in O(g(n))$$

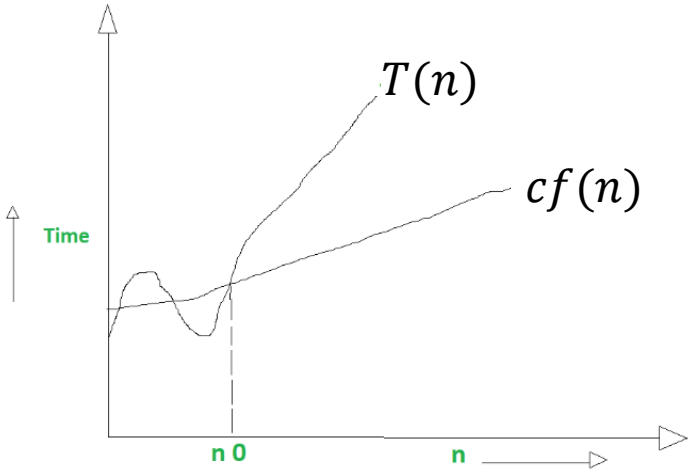
$$T(n) : \text{polynomial of degree } d \Rightarrow T(n) \in O(n^d)$$

Big-Omega & Big-Theta

- $T(n)$ is $\Omega(f(n))$ if there exist a constant $c > 0$ & an integer constant $n_0 \geq 1$ such that

$$T(n) \geq c \cdot f(n) \text{ for all } n \geq n_0$$

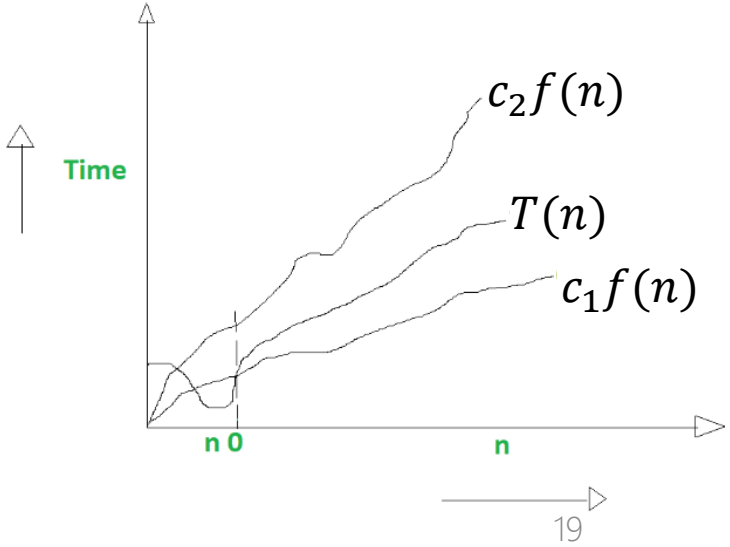
  - Lower bound - asymptotically greater than or equal to $f(n)$



- $T(n)$ is $\Theta(f(n))$ if $T(n)$ is $O(f(n))$ and $\Omega(f(n))$

$$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n) \text{ for all } n \geq n_0$$

  - Asymptotically equal to $f(n)$



Asymptotic Algorithm Analysis

- The asymptotic analysis of an algorithm determines the running time in big-Oh notation

1. Find the worst-case # of *primitive operations* executed as a function of the input size
2. Express it with big-O notation

Assumptions in Asymptotic Analysis

Basic operations take constant time

- Arithmetic (fixed-width)
- Assignment
- Access one array index
- Comparing two simple values (is $x < 3$)

Other operations are summations or products

- Consecutive statements are summed
- Loops are (cost of loop body) $\times$ (number of loops)

Asymptotic Algorithm Analysis: Example

program segment

```
for i:=1 to n do
    for j:=1 to n do begin
        C[i,j]:=0;
        for k:=1 to n do
            C[i,j]:=C[i,j]+A[i,k]*B[k,j]
    end
```

$$T(n) = \sum_{i=1}^{n}\sum_{j=1}^{n}(c_1 + \sum_{k=1}^{n} c_2) = \sum_{i=1}^{n}\sum_{j=1}^{n}(c_1 + c_2 \cdot n)$$

$$= \sum_{i=1}^{n}(c_1 \cdot n + c_2 \cdot n^2) = c_1 \cdot n^2 + c_2 \cdot n^3$$

$$\Rightarrow T(n) \in O(n^3)$$

Limitations of Asymptotic Analysis

- Not account for constant factors, but constant factor may dominate
  - $1000n$ vs. n^2 (when interested only in $n < 1000$)

- Not account for different memory access times at different levels of memory hierarchy
  - Cache Memory << MM << HDD

- Programs that do more computation may take less time than those that do less computation
  - Cost (fetch from main memory) >> Cost (operation in CPU)
  - Memory access could take more than computation

Intuition for Asymptotic Notation

- Big-O

  $f(n)$ is $O(g(n))$ if $f(n)$ is asymptotically less than or equal to $g(n)$

- Big-Omega

  $f(n)$ is $\Omega(g(n))$ if $f(n)$ is asymptotically greater than or equal to $g(n)$

- Big-Theta

  $f(n)$ is $\Theta(g(n))$ if $f(n)$ is asymptotically equal to $g(n)$

Quiz

- What if there is a “if” statement inside of an algorithm that is executed only at some iterations of a loop?
- How can you apply asymptotic analysis?

Now to the Board

My array is too small. Let's enlarge it.

Option 1: Increase array size by 10
Copy old array into new one

Option 2: Double the array size
Copy old array into new one

Stretchy Array (version 1)

$StretchyArray$:

$maxSize$: positive integer (starts at 1)

$array$: an array of size maxSize

$count$: number of elements in array

$put(x)$: add x to the end of the array

$if\ maxSize\ ==\ count$

$make\ new\ array\ of\ size\ (maxSize\ +\ 5)$

$copy\ old\ array\ contents\ to\ new\ array$

$maxSize\ =\ maxSize\ +\ 5$

$array[count]\ =\ x$

$count\ =\ count\ +\ 1$

Slides adopted from Kate Deibel

Stretchy Array (version 2)

$StretchyArray$:

$maxSize$: positive integer (starts at 0)

$array$: an array of size maxSize

$count$: number of elements in array

$put(x)$: add x to the end of the array

$if\ maxSize\ ==\ count$

$make\ new\ array\ of\ size\ (maxSize\ *\ 2)$

$copy\ old\ array\ contents\ to\ new\ array$

$maxSize\ =\ maxSize\ *\ 2$

$array[count]\ =\ x$

$count\ =\ count\ +\ 1$

Slides adopted from Kate Deibel

Performance Cost of $put(x)$

In both stretchy array implementations, $put(x)$ is defined as essentially:

$$
\begin{aligned}
&if\ maxSize\ ==\ count \\
&\quad make\ new\ array\ of\ bigger\ size \\
&\quad copy\ old\ array\ contents\ to\ new\ array \\
&\quad update\ maxSize \\
&array[count]\ =\ x \\
&count\ =\ count\ +\ 1
\end{aligned}
$$

What $f(n)$ is $put(x)$ in $O(\ f(n)\ )$?

Slides adopted from Kate Deibel

Performance Cost of $put(x)$

In both stretchy array implementations, $put(x)$ is defined as essentially:

$if\ maxSize\ ==\ count$	$O(1)$
$\quad make\ new\ array\ of\ bigger\ size$	$O(1)$
$\quad copy\ old\ array\ contents\ to\ new\ array$	$O(n)$
$\quad update\ maxSize$	$O(1)$
$array[count]\ =\ x$	$O(1)$
$count\ =\ count\ +\ 1$	$O(1)$

In the worst-case, $put(x)$ is $O(n)$ where n is the current size of the array!!

Slides adopted from Kate Deibel

But…

- We do not have to enlarge the array each time we call $put(x)$
- What will be the average performance if we put n items into the array?

$$\frac{\sum_{i=1}^{n} \textbf{cost of calling put for the ith time}}{n} = O(?)$$

- Calculating the average cost for multiple calls is known as *amortized analysis*

Slides adopted from Kate Deibel

Amortized Analysis of $StretchyArray$ Version 1

i	$maxSize$	$count$	$cost$	*comments*
	0	0		Initial state
1	5	1	$0 + 1$	Copy array of size 0
2	5	2	1	
3	5	3	1	
4	5	4	1	
5	5	5	1	
6	10	6	$5 + 1$	Copy array of size 5
7	10	7	1	
8	10	8	1	
9	10	9	1	
10	10	10	1	
11	15	11	$10 + 1$	Copy array of size 10
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Slides adopted from Kate Deibel

Amortized Analysis of $StretchyArray$ Version 1

i	$maxSize$	$count$	$cost$	*comments*
	0	0		Initial state
1	5	1	$0 + 1$	Copy array of size 0
2	5	2	1	
			1	
			1	
			1	
			$5 + 1$	Copy array of size 5
			1	
8	10	8	1	
9	10	9	1	
10	10	10	1	
11	15	11	$10 + 1$	Copy array of size 10
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Every five steps, we have to do a multiple of five more work

Slides adopted from Kate Deibel

Amortized Analysis of $StretchyArray$ Version 1

Assume the number of $puts$ is $n = 5k$

- We will make n calls to $array[count] = x$
- We will stretch the array k times and will cost:

$$0 + 5 + 10 + \ldots + 5(k-1)$$

Total cost is then:

$$n + (0 + 5 + 10 + \ldots + 5(k-1))$$
$$= n + 5(1 + 2 + \ldots + (k-1))$$
$$= n + 5(k-1)(k-1+1)/2$$
$$= n + 5k(k-1)/2$$
$$\approx n + n^2/10$$

Amortized cost for $put(x)$ is

$$\frac{n + \frac{n^2}{10}}{n} = 1 + \frac{n}{10} = O(n)$$

Slides adopted from Kate Deibel

Amortized Analysis of $StretchyArray$ Version 2

i	$maxSize$	$count$	$cost$	*comments*
	1	0		Initial state
1	1	1	1	
2	2	2	$1 + 1$	Copy array of size 1
3	4	3	$2 + 1$	Copy array of size 2
4	4	4	1	
5	8	5	$4 + 1$	Copy array of size 4
6	8	6	1	
7	8	7	1	
8	8	8	1	
9	16	9	$8 + 1$	Copy array of size 8
10	16	10	1	
11	16	11	1	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Slides adopted from Kate Deibel

Amortized Analysis of $StretchyArray$ Version 2

i	$maxSize$	$count$	$cost$	*comments*
	1	0		Initial state
1	1	1	1	
2	2	2	$1 + 1$	Copy array of size 1
3	4	3	$2 + 1$	
4	4	4	1	
5	8	5	$4 + 1$	
6	8	6	1	
7	8	7	1	
8	8	8	1	
9	16	9	$8 + 1$	Copy array of size 8
10	16	10	1	
11	16	11	1	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Enlarge steps happen basically when i is a power of 2

Slides adopted from Kate Deibel

Amortized Analysis of $StretchyArray$ Version 2

Assume the number of puts is $n = 2^k$

- We will make n calls to $array[count] = x$
- We will stretch the array k times and will cost:

$\approx (1 + 2 + 4 + \ldots + 2^{k-1})$

Total cost is then:

$\approx n + (1 + 2 + 4 + \ldots + 2^{k-1})$

$\approx n + 2^k - 1$

$\approx 2n - 1$

Amortized cost for $put(x)$ is

$$\frac{2n-1}{n} = 2 - \frac{1}{n} = O(1)$$

Slides adopted from Kate Deibel

The Lesson

With amortized analysis, we know that over the long run (on average):

- If we stretch an array by a constant amount, each $put(x)$ call is $O(n)$ time
- If we double the size of the array each time, each $put(x)$ call is $O(1)$ time

Slides adopted from Kate Deibel

Summary

- Algorithm
- Empirical comparison and asymptotic theoretical comparison
- Best, worst, average cases
- Big O notation
- Amortized analysis